Securing Modbus TCP Communications in I4.0: A Penetration Testing Approach Using OpenPLC and Factory IO

2023 IEEE Conference on Standards for Communications and Networking (CSCN) | 979-8-3503-9538-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/CSCN60443.2023.10453119

George Lazaridis
Information Technologies Institute (ITI)
Centre for Research & Technology Hellas & International Hellenic University
Thessaloniki, Greece
glazaridis@iti.gr

Anastasios Drosou
Information Technologies Institute (ITI)
Centre *for Research & Technology Hellas*
Thessaloniki, Greece
drosou@iti.gr

Periklis Chatzimisios
International Hellenic University, Greece & University of New Mexico, USA
pchatzimisios@ihu.gr

Dimitrios Tzovaras
Information Technologies Institute (ITI)
Centre for Research & Technology Hellas
Thessaloniki, Greece
dimitrios.tzovaras@iti.gr

***Abstract*— The advent of Industry 4.0 has brought forward transformative advancements in industrial automation and control systems, enhancing operational efficiency and productivity. Among the key communication protocols facilitating this paradigm shift, Modbus TCP is a widely adopted protocol for real-time data exchange between devices in industrial environments. However, the proliferation of interconnected devices also raises significant cybersecurity concerns. The current paper presents a comprehensive penetration testing approach aimed to securing Modbus TCP communications within the context of Industry 4.0. Leveraging the capabilities of software platforms, such as OpenPLC and Factory IO, we simulate industrial control systems to assess the vulnerabilities and weaknesses present in Modbus TCP implementations. By proactively addressing Modbus TCP security issues, our research work contributes to the safeguarding of critical infrastructure and underscores the importance of cybersecurity in the Industry 4.0 era.**

Keywords—Industry 4.0, Modbus TCP, penetration testing, simulation, cybersecurity, PLC

I. Introduction

In an era marked by the convergence of Information Technology (IT) and Operational Technology (OT) in the scope of Industry 4.0 (I4.0), the industrial landscape has witnessed unprecedented transformations. Industrial Control Systems (ICSs), once isolated and proprietary, have now become interconnected and increasingly reliant on standard networking protocols. Among these, Modbus TCP stands as a prominent choice for communication within industrial systems, offering efficiency and flexibility. However, the integration of Modbus TCP into the fabric of critical infrastructure has not come without its cybersecurity challenges.

The proliferation of interconnected devices and the advent of the Industrial Internet of Things (IIoT) have expanded the attack surface of industrial environments, rendering them susceptible to a spectrum of cyber threats. Securing Modbus TCP communications is of paramount importance, as any security compromise could lead to severe consequences, including unauthorized access, data manipulation and system downtime. Thus, it is imperative to thoroughly evaluate the security posture of Modbus TCP implementations in industrial systems.

The current research paper embarks on a journey to address the pressing need for enhanced cybersecurity in industrial systems. Leveraging the power of penetration testing, we explore the vulnerabilities and potential attack vectors that may threaten Modbus TCP communications. To conduct this research, we employ a robust, comprehensive but semi open-source testing software environment consisting of OpenPLC, Factory IO, pfSense and Kali Linux. Through this approach, we aim to assess the security landscape of Modbus TCP in industrial settings, identify weaknesses and propose a penetration testing strategy, with the ultimate goal of further researching on automating the penetration testing process by introducing Machine Learning (ML) techniques in order to simplify the whole procedure. With this approach, we endeavour to contribute to the protection of critical infrastructure, ensuring its resilience against the evolving threat.

The rest of the paper is organized as follows: Section II presents background information on Modbus TCP penetration testing together with the related work. Section III analyzes the industrial simulation environment, presenting all aforementioned tools that are jointly utilized. Section IV illustrates the conducted penetration testing procedures, taking advantage of the earlier-mentioned testbed and lists the findings, while Section V concludes the performed work and proposes potential future directions.

II. Background and Related work

The discussion about security for ICSs, particularly of Modbus TCP communications, has garnered significant attention in recent years due to the growing interconnectivity of industrial devices and the potential risks associated with cyberattacks on critical infrastructure. A wealth of research and practical solutions has emerged to address the unique security challenges posed by Modbus TCP and similar protocols. More specifically, authors in [1] demonstrate the potential of misuse-based Intrusion Detection Systems (IDSs) to detect and prevent attacks on Modbus/TCP-based Industrial Automation and Control Systems (IACSs). The proposed

orized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 04,2025 at 09:06:22 UTC from IEEE Xplore. Restrictions ap

approach is simple to be implemented and can be used to detect a wide range of attacks. Nevertheless, the authors of the work in [2] present a novel and effective approach to securing industrial communication protocols. This proposal, based on a widely used protocol, addresses several key security vulnerabilities and can be used to protect ICSs from various attacks.

Previous research efforts have delved into the security assessment of Modbus TCP in industrial settings. Notable studies have conducted vulnerability assessments, penetration tests and threat-modelling to uncover weaknesses and potential attack vectors. These assessments have revealed weaknesses of unauthorized access, command injection and data manipulation, and the need for advanced security. Authors in [3] present an approach for testing the security of Modbus TCP devices. The proposed tool, namely MTF, can easily be used to find a wide range of security vulnerabilities and by organizations to improve the security of their IACSs. On the other hand, authors in [4] propose a combined AI-based detection and SDN-based mitigation solution to defend the IIoT against Modbus TCP threats, while in [5] authors propose a novel method for enhancing the cybersecurity of Modbus TCP-based IACSs by implementing an authentication method based on Message Authentication Codes (MACs).

Penetration testing has also emerged as a crucial tool in assessing the cybersecurity of industrial systems. Researchers and practitioners have employed penetration testing methodologies to identify vulnerabilities in critical infrastructure components, including Programmable Logic Controllers (PLCs), Remote Terminal Units (RTUs) and Human-Machine Interfaces (HMIs). These assessments have provided valuable insights into the security of industrial environments, emphasizing the significance of rigorous testing and security audits. The authors in [6] investigate methods to secure RTUs in smart grid systems. RTUs are the building blocks of smart grids and are responsible for collecting and transmitting data from field devices to central control systems. However, RTUs are also vulnerable to a variety of cyberattacks, which could disrupt or disable the smart grid. Going one step further, researchers have already studied the possibility of integrating Artificial Intelligence (AI) into the process of penetration testing in order to make it more automated, smarter and less time consuming. For this reason, authors in [7] present a comprehensive overview of the state-of-the-art, and the benefits and the research challenges of AI-enabled IoT penetration testing.

The use of industrial simulation environments, such as OpenPLC and Factory IO, has gained prominence for evaluating ICS security. These platforms enable researchers to create realistic industrial scenarios, replicate system configurations and conduct controlled experiments. Studies have demonstrated the effectiveness of such environments in assessing the security of industrial protocols, making them invaluable tools for security researchers. An example of a developed industrial testbed is presented in [8], where authors describe a novel testbed for cybersecurity analysis of Nuclear Power Plants (NPPs). The testbed is open-source and based on the Modbus protocol. It can be used to simulate a realistic NPP environment and to test the security of ICS devices and networks. Besides the aforementioned testbed, authors in [9] survey a wide range of ICS testbeds, including testbeds that are developed by academia, industry or government agencies focusing on the increasing use of open-source software and hardware, the increasing use of cloud computing to deploy and manage these testbeds and finally the increasing use of AI and ML to improve the effectiveness of ICS testbeds.

III. Simulation Environment

One of the most significant challenges in the industrial cybersecurity sector is the prohibitive cost of purchasing the devices, together with any software licensing, required to gain practical experience working with OT systems and processes. Virtualization is an excellent method to overcome this obstacle and start becoming familiar with the protocols and characteristics of the ICS world with only a relatively small primary cost.

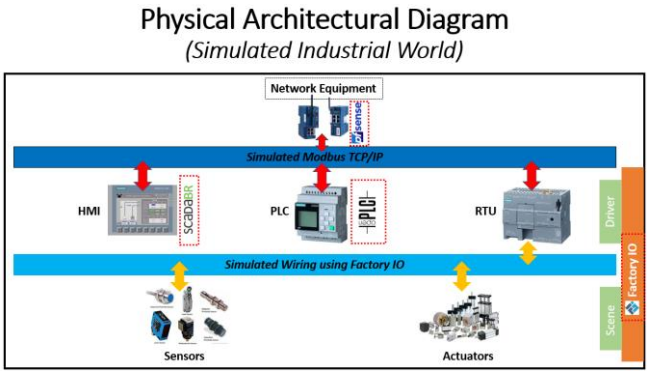


Fig. 1. Physical architectural diagram of simulated environment

Figure 1 depicts the simulated industrial environment that consists of the following components, all of which were installed on a Windows 11 host machine (Intel Core i5-6600 CPU @ 3.30Ghz, 40GB RAM, 2x 1TB SSD), either as Virtual Machines (VMs) or straight installations on the host:

- pfSense: Software for simulating the network equipment
- SCADABR: Software for simulating the HMI
- OpenPLC: Software for simulating the PLC
- OpenPLC Editor: Complementary software in order to program the PLC activities
- Factory IO: Software for simulating the RTU (driver) and the sensors & actuators (scene)
- Kali Linux: Operating System (OS) simulating the malicious actions against the Modbus TCP simulation environment

A. *pfSense*

pfSense is an open-source, FreeBSD-based firewall and routing software that has gained significant popularity in the field of network security and management. It was initially developed as a fork of the m0n0wall project [10] in 2004 and has evolved into a robust and versatile solution for a wide range of networking needs. With flexibility and scalability, it suits businesses, schools, and individuals. pfSense boasts a rich feature set that caters to diverse network requirements. Core features include NAT, VPN, and traffic shaping, managed via an intuitive web interface. pfSense excels at Quality of Service (QoS) for bandwidth optimization and supports packages like IDS/IPS, content filtering, and proxy services. In the context of our testbed, pfSense runs in a VM and created two network interfaces as depicted in Figure 2.

orized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 04,2025 at 09:06:22 UTC from IEEE Xplore. Restrictions ap

B. *SCADABR*

SCADABR, a versatile open-source SCADA system, aids real-time process monitoring. Its web interface, historical data, and customizability attract users for cost-effective SCADA solutions. In our testbed system runs on a separate Ubuntu Server VM, connected via pfSense's network interface.

C. *OpenPLC*

OpenPLC, a flexible open-source PLC platform, aids industrial automation. It supports diverse hardware, languages, and protocols, plus simulation. Test system is running on Ubuntu Server VM and is connected to pfSense and SCADABR for seamless communication.

D. *OpenPLC Editor*

OpenPLC Editor is an Integrated Development Environment (IDE) for OpenPLC, streamlining control logic program creation and management. It simplifies design and deployment for automation engineers. This software tool does not run in a VM (runs straight on the host machine), complementary to OpenPLC. It supports various programming languages and is integral to programming PLCs compliant with IEC 61131-3.

E. *Factory IO*

Factory IO, an interactive simulation software, is ideal for industrial automation training. It creates a virtual environment (Figure 3) for users to experiment with scenarios. It's a commercial program, installed on the host machine, communicating via pfSense, as shown in Figure 2.

F. *Kali Linux*

Kali Linux is essential for cybersecurity experts and enthusiasts. It's purpose-built with a rich toolset and strong community support for ethical hacking and penetration testing. In the context of this research, it is installed in a separate VM, communicating with the other VMs through the network interface.

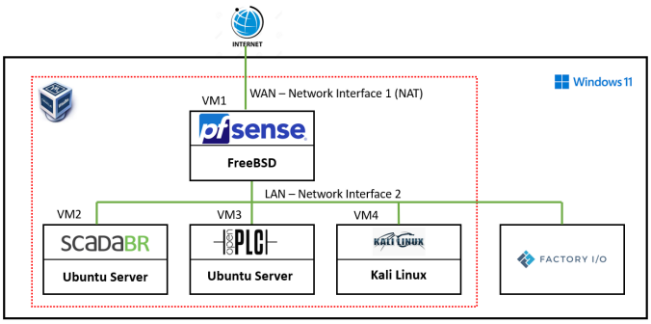


Fig. 2. Simulated industrial environment depicting all used software



Fig. 3. Factory IO screenshot of the simulated production line

The simulation setup unfolds as follows: Firstly, the pfSense VM is activated to establish two crucial network interfaces – one for VM-to-VM communication and one for internet access. Next, the VM with OpenPLC will boot, simulating a PLC communicating via Modbus TCP. OpenPLC Editor is utilized to upload the firmware via a function block diagram. The Factory IO simulation platform then starts, which represents a RTU for handling actuators and sensors. Scene should be uploaded and Modbus TCP communication between OpenPLC (client) and Factory IO (server) should be configured. The simulation initiates and displays the production line. Lastly, Kali Linux is executed to investigate Modbus TCP protocol vulnerabilities in the simulated industrial environment for cybersecurity purposes.

IV. Penetration Testing and Findings

Industrial penetration testing is a specialized cybersecurity practice focused on assessing the security of critical infrastructure and industrial control systems used in sectors like energy or manufacturing. It involves ethical hackers, often with expertise in industrial automation, conducting systematic tests to identify vulnerabilities, weaknesses and potential entry points within ICS environments. The primary goal is to proactively uncover security flaws that could lead to disruptions, unauthorized access or safety risks, allowing organizations to strengthen their defences and protect vital industrial processes and assets from cyber threats.

There are five penetration testing phases, namely reconnaissance, scanning, vulnerability assessment, exploitation and reporting. In some cases, there might be even more penetration testing phases, e.g., a pre-engagement interactions phase, but usually they are five. On the other hand, three different penetration testing approaches occur, the white box, the grey box and the black box. In the current work, the selected penetration testing approach is the white box, assuming that the tester has comprehensive knowledge of the target system, including internal architecture, source code and configurations.

In this paper, the primary emphasis will be placed on the initial stage of penetration testing, which is the reconnaissance phase, also known as information gathering. This phase is very important becuase it serves as the cornerstone for obtaining critical insights. This information enables researchers to comprehensively understand the production environment and map its operations and subsequently execute targeted attack strategies. In most cases, it is possible that the attacker will employ a specialized hacking OS, such as Kali Linux or Parrot OS. Consequently, the selection of Kali Linux not only replicates the environment used by an actual attacker, thereby introducing their toolset to researchers, but also furnishes them with the requisite software tools. These tools enable cybersecurity personnel to proficiently oversee and interpret the communication and data exchanges transpiring within the targeted systems.

The initial step involves the identification of our position within the current subnet or network. Instead of havingh a simulation testbed and possessing comprehensive knowledge of the system's components, the best practice is to gain a thorough understanding of the network we are operating within. This not only helps identifying our target systems but also allows a carefull examination of the entire network or

orized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 04,2025 at 09:06:22 UTC from IEEE Xplore. Restrictions ap

subnet, with the objective of identifying potential vulnerabilities or tools that could serve as the attack vectors.

To accomplish this activity, we employ the terminal console of a Kali Linux distribution and execute the "ifconfig" command. This command serves the purpose of retrieving the IP address and examining the subnet mask. Our observation reveals that the subnet mask is configured as *255.255.255.0*, indicating that we are presently situated within a Class C network, as denoted by the Classless Inter-Domain Routing (CIDR) notation of /24. This particular configuration implies that the network used has the capacity to accommodate up to 254 assets, which may include computers, production machinery, servers or other industrial devices.

With this initial and valuable information in hand, the next step involves the utilization of Nmap, a versatile tool employed for host and service discovery on computer networks. Nmap achieves this by sending packets and meticulously analyzing the ensuing responses. It may be tempting to consider this step unnecessary, given that we are familiar with the simulated network topology, as already mentioned. However, this assumption is not accurate and the deployment of Nmap holds immense value, as it allows us to precisely identify the production line components actively interacting within the network. This, in turn, simplifies our role as attackers, allowing us to effectively monitor the data exchanges transpiring between these components. To initiate this process, we execute the following command within our terminal: "*nmap 192.168.88.0/24*". The outcome of this command outputs four IP addresses, all of which constitute integral parts of our simulated environment.

With a single command, the essential information about our network configuration was successfully acquired. The performed analysis reveals the presence of four assets (SCADABR is not part of this simulation) within our simulated network and it is important to identify each component accurately. We can presume that our own IP address is *.207* based on the "*ifconfig*" command previously executed. Furthermore, the identification of ports *53* (DNS), *80* (HTTP) and *443* (HTTPS) allows us to ascertain that the device requiring DNS services is our pfSense router. By the process of elimination, we can narrow our focus to the remaining two machines of interest. However, none of them appears to be the production line devices utilizing the Modbus protocol. It is important to note that Nmap, by default, scans the most commonly used ports for each protocol. Consequently, Modbus TCP port 502 was only scanned after the Nmap command was configured correctly. This specific configuration is depicted below:

$ sudo nmap -sT -sV -O -A -vv -p- [IP]

The aforementioned command is executed for both IPs 192.168.88.100 and 192.168.88.201. The results are depicted in Figures 4 and 5:



Fig. 4. Nmap execution for 192.168.88.100 showing open ports



Fig. 5. Nmap execution for 192.168.88.201 showing open ports

The process of recognition has become significantly more straightforward. It is now evident that the asset with the IP address *192.168.88.100* is the machine of interest. This assumption can be drawn with confidence as Factory IO is exclusively deployed and executed in a Windows environment. Moreover, the presence of port *502,* utilized for Modbus communications but also for various other Windows services, leaves no room for ambiguity. Consequently, the asset assigned to the IP address *192.168.88.201* can logically be assigned to the OpenPLC software. This deduction is grounded by several key factors. Firstly, OpenPLC operates on a Linux-based system, distinguishing it from other components. Secondly, the utilization of port *502* aligns with Modbus communications. With this information, we can now confidently note which machine corresponds to each specific software component.

Once the IP addresses of each VM were identified and confirmed, since the followed approach is the white box, Wireshark was executed in order to capture some exchanged packets between the OpenPLC and the Factory IO. Wireshark was executed directly from Kali Linux and the capturing of the network traffic of interface eth0 started, which is in promiscuous mode and therefore able to monitor all network traffic. As we are only interested in Modbus communication, we will use the filter "*tcp.port==502*". This will allow us to follow all exchanged data between these two components. The communication begins with the standard 3-way handshake, following the usual TCP connection patern. It is important to understand that the machine initiates the communication, in order to distinguish the server from the client. Since the one that has the production lines' instructions and transmits them, is usually the OpenPLC, we expect that it is the one sending the first *SYN* to Factory IO. Factory IO replies with the standard *SYN/ACK,* and as the last step of the handshaking, OpenPLC responds with an *ACK,* so the communication channel is ready to be used.

From an early stage, it can be noticed that there is a loop in the communication, which is described and appears to be well-documented in the "*MODBUS Messaging on TCP/IP Implementation Guide*" [11]. In the scenario that we consider in the current work, Modbus uses two standard functions: the "*Read Discrete Inputs"* and the "*Write Multiple Coils"*. Each of the repetitions consists of the query, the response and the ACK part. Having this information in mind, allows us to monitor the communication easier and more effectively. In the scope of the current work, we will examine carefully a "*Read Discrete Inputs*" and a "*Write Multiple Coils*" request.

A. Read Discrete Inputs

The following information can be extracted from the traffic recording in Wireshark:

orized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 04,2025 at 09:06:22 UTC from IEEE Xplore. Restrictions ap

1. Regarding the source and destination ports, since OpenPLC initiates the communication, it uses a random port, while the FactoryIO which is only listening, is waiting for instructions on port *502*.
2. For both request and response, the same Transaction ID is used.
3. For the first time, we are obtaining the RTU's ID number. In our simulation, the ID is constantly "*1*", but in a real-life production environment, this number may vary or even have multiple IDs for multiple RTUs.
4. For both request and response, it is observed that the function code is "*000 0010"*, which is equal to decimal "*2*". This function code in Modbus actually corresponds to "*Read Discrete Input*".
5. Finally, the request requires 16 bits (8 and 8 bits) starting at position 0 (as a reference number). In our testing environment, we configured OpenPLC and FactoryIO to use two production lines (Figure 3), each of 8 total inputs. So, the first number (query digits 1 and 2), refers to the inputs of the first production line, while the second one (query digits 3 and 4), refers to the inputs of the second production line.

To this end, numbers 00_{16} 44_{16} are observed and we know that the first number refers to the 1st production line. As a first step, we convert the number to binary, so we have 00_{16} <==> 00000000_2. We know that the first bit starts at position 0, so we have the following bits in the following positions:

Position	0	1	2	3	4	5	6	7
Bits	0	0	0	0	0	0	0	0

We apply the same technique for the second production line. First converting the 44_{16} to binary, gives us the following bits: 01000100_2

Position	0	1	2	3	4	5	6	7
Bits	0	1	0	0	0	1	0	0

The mapping shown above reveals the status of each input (1=on, 0=off), however we cannot retrieve the information of which input corresponds to which sensor. Therefore, this issue will be examined in one of the next steps. At this point we need the assistance of OpenPLC. From the captured values and by using the registers, we can understand which sensor corresponds to each component in our production lines. For example, if we have a payload of 0111011001000000_2, we can map it as following:

Register	%IX100.0	%IX100.1	%IX100.2	%IX100.3	%IX100.4	%IX100.5	%IX100.6	%QX100.7	%IX101.0	%QX101.1
Bit	0	1	1	1	0	1	1	0	0	1

Observing the above table, we can understand what is the exact state of all our sensors. It is worth mentioning, that the last 6 bits are not used, because in our schema, both production lines use only 10 sensors. Yet, we still do not know which value corresponds to each component in Factory IO. The assistance of OpenPLC will allow us to map the registers with the point names:

I_Z01BC01ArrivedSens	BOOL	%IX100.0	No
I_Z01BC02EntrySens	BOOL	%IX100.1	No
I_Z01MC01_Open	BOOL	%IX100.2	No
I_Z01MC01_Bussy	BOOL	%IX100.3	No

Fig. 6. Point names as illustrated in the OpenPLC program

Finishing the mapping process, we are able to tell the exact location of the sensors placed in the production line. From the descriptive point names, we can always assume what are the performed action. However, we need to understand which sensor belongs to each production line. In order to perform this task, we need to correspond the point names with the actual components, by using the Factory IO and matching the values. By the end of this task, we will know with certainty how sensors are correlated with our simulated environment.

B. Write Multiple Coils

The same applies in this Modbus function, meaning that the following information can be extracted from the traffic recording in Wireshark:

1. For the source and destination ports, the same assumption is made as in the previous function (Read Discrete Inputs): OpenPLC initiates the communication using a random port and then FactoryIO who is listening, awaits instructions on port *502*.
2. For both, request and response, the same Transaction ID is used.
3. In this field, we are expecting to see the RTU's ID number, which is 1, but -as already mentioned- in an actual production environment, that number may vary or even have multiple IDs for multiple RTUs.
4. For both, request and response, we can observe that the function code is "*000 1111*", which equals to decimal "*15*". This function code in Modbus actually corresponds to "*Write Multiple Coils*".
5. This function requires 14 bits (7 and 7 bits) starting at 0 (as a reference number). In this testbed, we configured OpenPLC and Factory IO to use 2 production lines, each consisting of 7 total outputs. To this end, the first number refers to the outputs of the first production line, while the second one refers to the outputs of the second production line.

In this case, we need to examine different frames of the captured traffic, to better understand the use of the actuators. This will allow us to understand the changes as they occur, and map them, in order to find out which actuator corresponds to which component in Factory IO. In the following table, the output for both production lines can be summarized:

Register	%QX100.0	%QX100.1	%QX100.2	%QX100.3	%QX100.4	%QX100.5	%QX100.6	%QX100.7	%QX101.0	%QX101.1	%QX101.2	%QX101.3	%QX101.4	%QX101.5	%QX101.6
Bit	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0

Now, the exact state of all our actuators is known. But we still do not know which value corresponds to each component. We can use the assistance of OpenPLC again, allowing us to map the registers with the point names, as already shown in Figure 6.

orized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 04,2025 at 09:06:22 UTC from IEEE Xplore. Restrictions ap

Having analyzed all Modbus information and with the usage of the information collected by Factory IO, we are able to distinguish which actuators are in which production line and performed all corresponding activities. At this point, it is worth mentioning that in all Modbus communications (TCP, RTU) there are no security mechanisms implemented, meaning that someone could have full access to the information allowing a malicious actor to read and write all data in Modbus communications. Consequently, the packet sniffing with Wireshark revealed several pieces of information that could be used to compromise the Modbus TCP communication.

Based on the information gathered using the Wireshark tool and taking advantage of the known information on the sensors and actuators in Factory IO, also including the mapping of the components used in the scene, we are able to recognize the packet sequences they exchange. By listing the exact bits they receive, the security researcher had the ability to intervene and execute a Man-in-The-Middle (MiTM) attack and inject different bits than those sent regularly. This action was conducted using the Metasploit framework of Kali. More specifically, it provides a Modbus tool that allows the user, to read the registers and then perform the intervention (write function) of a different bit. This leads to the malfunction of the production line, causing several damage and financial loss. Besides Metasploit, another Modbus penetration testing tool was examined, namely the Smod tool, but the attacks conducted through this tool did not affect the operations of the testbed at all.

V. CONCLUSIONS

The current work presented the initial work of our cybersecurity research in industrial environments, aiming at the development of an industrial AI-based automated penetration testing module that can be easily used by anyone, without the need of any particular cybersecurity knowledge. Towards this direction, we first deployed a industrial simulated environment to be able to simulate modbus communications and then we gathered the necessary information in order to perform the penetration tests. The first results reveal that the Modbus TCP protocol can be easily compromized by performing only a simple attack by injecting information different from the information expected. The findings of these tests where illustrated in the simulation environment of Factory IO, showing that the production line was malfunctioning. On the other hand, the Smod tool failed to perform any attacks towards the simulation evironment.

The next steps of our research journey include the development of an IEC 61499-based automation platform that supports *Modbus TCP*, *MQTT* and *OPC-UA* communication protocols that will allow the examination of cybersecurity activities in a more realistic environment closer to an I4.0 scene. The penetration testing activities described in the current research paper will be further developed, taking advantage of ML algorithms that will automate the procedures and itroduce intelligence that will eliminate the human intervention in some steps of the penetration testing process. The final step will include the deployment of the industrial AI-based automated penetration module in a real industrial environment in the context of the research project funding this work. This will allow us to draw conclusions of the proposed algorithm in a real environment and examine as many as possible communication protocols.

ACKNOWLEDGMENT

This work has received funding from the European Union's Horizon Europe research and innovation programme under grant agreement No. 101057083 – Zero-SWARM

REFERENCES

[1] F. Katulić, D. Sumina, I. Erceg and S. Groš, "Enhancing Modbus/TCP-Based Industrial Automation and Control Systems Cybersecurity Using a Misuse-Based Intrusion Detection System", in Proceedings of the International Symposium on Power Electronics, Electrical Drives, Automation and Motion (SPEEDAM), Sorrento, Italy, pp. 964-969, 2022.

[2] W. Jingran, L. Mingzhe, X. Aidong, H. Bo, H. Xiaojia and Z. Xiufang, "Research and Implementation of Secure Industrial Communication Protocols", in Proceedings of the IEEE International Conference on Artificial Intelligence and Information Systems (ICAIIS), Dalian, China, pp. 314-317, 2020.

[3] A. Voyiatzis , K. Katsigiannis, and S. Koubias,. "A Modbus/TCP fuzzer for testing internetworked industrial systems", in Proceedings of the 20th IEEE Conference on Emerging Technologies & Factory Automation (ETFA 2015), pp. 1–6, 2015.

[4] P. Radoglou-Grammatikis et al., "Defending Industrial Internet of Things Against Modbus/TCP Threats: A Combined AI-Based & Detection and SDN-Based Mitigation Solution," SSRN Electronic Journal, 2022.

[5] F. Katulić, D. Sumina, S. Groš and I. Erceg, "Protecting Modbus/TCP-Based Industrial Automation and Control Systems Using Message Authentication Codes", in IEEE Access, vol. 11, pp. 47007-47023, 2023.

[6] E. R. Ling, J. E. Urrea Cabus, I.Butun, R. Lagerström, and J. Olegard, "Securing Communication and Identifying Threats in RTUs: A Vulnerability Analysis", in Proceedings of the 17th International Conference on Availability, Reliability and Security (ARES 2022). pp. 1–7, 2022.

[7] Greco Claudia, Fortino Giancarlo, Crispo Bruno, Choo Kim-Kwang Raymond "AI-enabled IoT penetration testing: state-of-the-art and research challenges", Enterprise Information Systems, vol. 17, issue 9, pp. 1223–1247, 2023.

[8] I. B. de Brito and R. T. de Sousa Jr., "Development of an open-source testbed based on the modbus protocol for cybersecurity analysis of nuclear power plants", MDPI Applied Sciences, vol. 12, no. 15, 2022.

[9] H. Holm, M. Karresand, A. Vidström & E. Westring, "A survey of industrial control system testbeds", in Proceedings of the Nordic Conference on Secure IT Systems (NordSec 2015), pp. 11–26, 2015.

[10] The m0n0wall project (https://m0n0.ch/wall/index.php)

[11] MODBUS Messaging on TCP/IP Implementation Guide V1.0b (https://www.modbus.org/docs/Modbus_Messaging_Implementation_Guide_V1_0b.pdf)

orized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 04,2025 at 09:06:22 UTC from IEEE Xplore. Restrictions ap